

Fiche de rappel 1 - Des besoins aux cardinalités

À utiliser après les leçons 1 à 3. Pour chaque symbole, reconstruire une phrase avec un objet précis et les règles du domaine.

Besoins et modèles : trois questions différentes

User story : qui souhaite faire quoi, et pourquoi ? Critère d'acceptation : dans quelle situation, quel résultat vérifiable ? Règle de gestion : quelle obligation faut-il toujours respecter ? Ces outils alimentent la conception Merise sans être eux-mêmes des modèles Merise.

Vocabulaire du MCD, modèle conceptuel de données

Mot	Lecture concrète
Entité	Type d'objet : LIVRE.
Occurrence	Un objet précis : le livre 7, Le Phare.
Attribut	Une propriété : titre.
Identifiant	Ce qui distingue chaque objet : livre_id.
Association	Un lien : un livre possède une copie.
Dictionnaire	Sens et valeurs autorisées des données.

Décoder minimum, maximum et n

Dans 0,n, le premier nombre est le minimum et le second le maximum. n indique plusieurs liens possibles, sans nombre fixé par cette notation ; il n'impose pas plusieurs liens présents. On lit la cardinalité près de l'entité depuis une seule occurrence de cette entité.

Quatre traductions en français

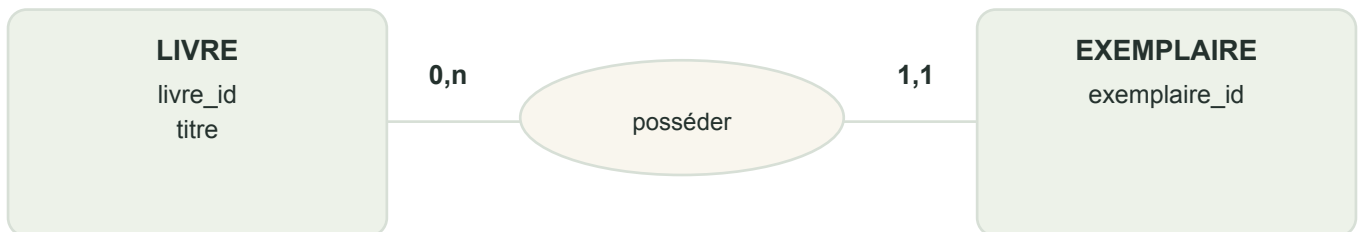
Cardinalité	Nombre de liens autorisés
0,1	Zéro ou un.
1,1	Exactement un.
0,n	Zéro, un ou plusieurs.
1,n	Un ou plusieurs.

Pour approfondir : [13] [01] [04] · Références détaillées à la fin.

SUITE DE LA FICHE

Fiche de rappel 1 - Des besoins aux cardinalités

LIVRE (0,n) - posséder - EXEMPLAIRE (1,1)



Un livre : de zéro à plusieurs exemplaires.

Un exemplaire : exactement un livre.

Lire les deux côtés, séparément

Pour un livre précis : zéro, une ou plusieurs copies. Pour une copie précise : exactement un livre. Le livre 8 sans copie est valide ; une copie sans livre ne l'est pas. Ne lire qu'un côté laisse une partie des règles inexprimée.

Binaire et ternaire

Binaire : deux types d'entités dans un fait, comme livre-copie. Ternaire : trois dans le même fait, comme enseignant-matière-classe. Avec Alice-SQL-C1, Alice-JavaScript-C2 et Bob-SQL-C2, les paires peuvent inventer Alice-SQL-C2. Conserver le triplet ou prouver que les relations séparées reconstruisent exactement les faits : c'est la reconstruction sans perte.

Pour approfondir : [13] [01] [04] · Références détaillées à la fin.

Fiche de rappel 2 - Construire les tables et les contraintes

À utiliser après les leçons 4 et 5. Les notations 1:n, n:n et 1:1 résument les maximums ; vérifier aussi les minimums du MCD.

Trois modèles à distinguer

Sigle et nom	Décision prise
MCD - modèle conceptuel de données	Quels objets et liens décrivent les règles ?
MLD - modèle logique de données	Quelles tables, colonnes et clés les représentent ?
MPD - modèle physique de données	Quels types et contraintes dans le moteur choisi ?

Lire la notation des clés

PK signifie clé primaire : une ou plusieurs colonnes identifient chaque ligne sans doublon ni absence. FK signifie clé étrangère : une valeur référence une clé existante dans une autre table. Une clé composée utilise plusieurs colonnes ensemble : le couple commande-produit distingue une ligne d'achat si le même produit ne se répète pas dans une commande.

Quatre raisonnements de traduction

Règle	Table et justification
Un livre peut avoir plusieurs copies. Une copie a exactement un livre.	Mettre livre_id dans EXEMPLAIRE : chaque copie a une seule référence.
Commandes et produits : plusieurs possibles de chaque côté.	Créer LIGNE avec une référence vers chaque table. Le couple identifie le lien ; quantité décrit ce lien.
Une personne a zéro ou un dossier. Un dossier a une personne.	Mettre personne_id dans DOSSIER : référence existante, obligatoire et non répétée.
Enseignant-matière-classe : seul le triplet est unique.	Créer AFFECTATION avec trois références. Les trois colonnes ensemble sont la clé.

Pour approfondir : [13] [31] · Références détaillées à la fin.

SUITE DE LA FICHE

Fiche de rappel 2 - Construire les tables et les contraintes

Contraintes PostgreSQL : relier un mot à une obligation

Clause	Obligation
PRIMARY KEY	Identifiant présent et unique.
REFERENCES	La valeur référencée existe.
NOT NULL	La valeur est présente.
UNIQUE	La valeur ou le couple soumis à cette contrainte ne se répète pas.
CHECK (quantite > 0)	Refuse une quantité négative ou nulle. Ajouter NOT NULL pour interdire une valeur absente.

Tester mentalement le cas 1:1

Les personnes 1 et 2 existent. DOSSIER contient (id = 10, personne_id = 1), avec personne_id déclaré REFERENCES, NOT NULL et UNIQUE. Un dossier supplémentaire pour 1 est refusé par UNIQUE ; un dossier pour 3, par REFERENCES ; un dossier sans personne, par NOT NULL. Un premier dossier pour 2 est accepté.

Limite à vérifier

La FK de LIGNE vers COMMANDE garantit que la commande référencée existe. Elle ne garantit pas qu'une commande possède au moins une ligne. Ce minimum demande une règle et un mécanisme supplémentaires. De même, l'historique de prêts répétés ne peut pas toujours être identifié par le seul couple lecteur-exemplaire : préciser ce qui distingue chaque prêt.

Pour approfondir : [13] [31] · Références détaillées à la fin.

Fiche de rappel 3 - Dépendances, normalisation et UML

À utiliser après les leçons 6 à 8. Les flèches de dépendance de données et les flèches UML se lisent avec des règles différentes.

Décoder $X \rightarrow Y$ et les clés

X et Y désignent des colonnes ou ensembles de colonnes. $X \rightarrow Y$ se lit « X détermine Y » : mêmes valeurs de X impliquent mêmes valeurs de Y dans tout état valide. Une superclé distingue chaque ligne. Une candidate est une superclé minimale. La primaire est une candidate choisie. Un attribut premier appartient à au moins une candidate.

Retrouver le problème avant de nommer la forme

Forme	Exemple et correction
1NF	Téléphones : 0601, 0602 dans une cellule. Choisir le domaine « un numéro » puis créer une ligne par numéro.
2NF	Clé de LIGNE : commande-produit. produit $\rightarrow$ libellé utilise seulement une partie. Séparer PRODUIT et LIGNE.
3NF	commande $\rightarrow$ client $\rightarrow$ nom. Le nom se répète dans chaque commande. Séparer CLIENT et COMMANDE.
BCNF - approfondissement	professeur $\rightarrow$ matière, mais professeur ne distingue pas chaque affectation scolaire. Séparer et vérifier les règles restantes.

Repères formels, après les exemples

Dans ces critères, X désigne un ensemble de colonnes et A un seul attribut. 2NF : 1NF, puis aucun attribut non premier ne dépend d'une partie stricte d'une candidate. 3NF : pour $X \rightarrow A$, si A n'appartient pas à X , X doit être une superclé ou A doit être premier. BCNF supprime cette dernière exception : X doit être une superclé.

Deux contrôles après séparation

Reconstruction sans perte : les tables permettent de retrouver exactement les lignes initiales. Conservation des dépendances : les règles peuvent être vérifiées dans les tables séparées sans les réunir. Les deux propriétés ne sont pas équivalentes. Ajouter un identifiant artificiel ne supprime pas les anciennes dépendances ni les autres clés candidates.

Pour approfondir : [27] [04] [30] · Références détaillées à la fin.

SUITE DE LA FICHE

Fiche de rappel 3 - Dépendances, normalisation et UML

UML et Merise : retrouver la question

Un cas d'utilisation UML décrit un service rendu à un acteur extérieur. Le MCD Merise décrit des données et leurs liens. Pour le distributeur, l'acteur est Client ; le cas de base est Retirer. Le code est vérifié dans chaque retrait ; le reçu n'est imprimé que sur demande et le retrait est complet sans lui.

Sens des flèches dans cet exemple

Lien UML	Flèche et lecture
include	Retirer → Vérifier le code : le cas de base utilise le comportement inclus.
extend	Imprimer le reçu → Retirer : le comportement ajouté complète la base sous condition, à un point prévu.

Le dessin doit être accompagné

Préciser les conditions de départ, les étapes, les alternatives et le résultat. Une flèche include ou extend n'exprime pas simplement une chronologie. Une user story exprime un besoin ; UML développe le service ; la conception des données se poursuit avec Merise.

Pour approfondir : [27] [04] [30] · Références détaillées à la fin.

TypeScript · mémo expliqué

Un mémo rappelle un raisonnement déjà travaillé. Commencer par dire quelles valeurs sont autorisées, puis ce qui sera réellement vérifié.

Descriptions de données

Écriture	Lecture et exemple
number / string / boolean	Nombre / texte / vrai ou faux : 2, "Lire", true.
nom: string ; pages?: number	nom textuel obligatoire ; pages facultatif.
string[] ; [string, number]	Liste de textes ; tuple texte en 0, nombre en 1.
string null	Union : texte OU absence volontaire.
interface / type	Nommer une forme ; ne crée aucune valeur.
readonly	Interdit certaines modifications au vérificateur.

Vérifier une entrée

unknown signifie qu'on ignore encore le type. `typeof value === "string"` justifie une opération de texte dans cette branche. Une union discriminée utilise un champ comme status pour distinguer les variantes et leurs propriétés. `any` retire des contrôles ; `as` affirme une hypothèse sans vérifier ni convertir. Une quantité valide demande aussi la vérification de son domaine : entier, minimum, maximum.

Réutilisation et attente

Écriture	Pourquoi et résultat
<code>premier<T>(liste: T[]): T undefined</code>	Conserve le type des éléments ; prévoir la liste vide.
<code>Promise<string></code>	Engagement dont le succès fournit un texte.
<code>async / await / try...catch</code>	Retourne une Promise / attend / traite un échec.
<code>export / import / import type</code>	Partager / récupérer / récupérer seulement un type.
<code>Partial / Pick / Omit</code>	Facultatif / sélectionner / retirer dans une description.
<code>Readonly / Record</code>	Interdire la réaffectation / décrire un dictionnaire.

Pour approfondir : [48] [49] [53] [51] [54] [50] [52] [55] [28] · Références détaillées à la fin.

SUITE DE LA FICHE

TypeScript · mémo expliqué

Comparer les deux moments

Avant le lancement : tsc vérifie le code, et strict rend ses contrôles plus exigeants. Pendant le lancement : le JavaScript manipule les données réelles ; les annotations ont disparu. tsx exécute un fichier TypeScript mais ne remplace pas tsc. Un résultat extérieur doit toujours être validé. Un type d'erreur ne garantit pas qu'une opération réussira.

Terminal · essai valeurs.ts de la première leçon

```
npx tsc --strict --noEmit valeurs.ts  
npx tsx valeurs.ts
```

Routine de lecture

- Séparer : la valeur écrite après = et son type annoncé après :.
- Pour |, énumérer les possibilités puis choisir une branche avec un test.
- Pour <T>, suivre le type d'entrée jusqu'au type de sortie.
- Avant await, on possède une Promise ; après réussite, sa valeur.
- Un utilitaire transforme une description, pas un objet reçu.

Pour approfondir : [48] [49] [53] [51] [54] [50] [52] [55] [28] · Références détaillées à la fin.

SQL · mémo des étapes et des garanties

SQL signifie Structured Query Language. Pour lire une requête, suivre les lignes ; pour écrire, identifier la cible et la décision de validation.

Structure et intégrité

Terme développé	Garantie ou exemple
PK · Primary Key, clé primaire	Identité unique et présente de chaque ligne.
FK · Foreign Key, clé étrangère	Une référence renseignée doit exister. Plusieurs enfants possibles.
NULL / NOT NULL	Absence ou valeur inconnue / valeur obligatoire.
UNIQUE	Interdit les doublons selon la contrainte.
CHECK	Condition vraie ou inconnue acceptée ; ajouter NOT NULL si requis.

Lecture de lignes et de groupes

Clause ou fonction	Question à poser
FROM / WHERE / SELECT	Source / lignes gardées / colonnes montrées.
ORDER BY / LIMIT / OFFSET	Ordre défini / combien / combien ignorer.
INNER JOIN / LEFT JOIN / ON	Correspondances seules / garder la gauche / condition du lien.
GROUP BY / COUNT(col) / SUM	Former les groupes / compter les non-NULL / additionner.
HAVING / COALESCE	Filtrer les groupes / premier résultat non-NULL.
IS NULL / EXISTS	Absence ? / au moins une ligne dans la sous-requête ?

Le cas Alice et Bilal

Données : Alice possède les commandes 10/30 et 11/20 ; Bilal n'en a aucune. LEFT JOIN donne deux lignes pour Alice et une ligne sans commande pour Bilal. COUNT(co.id) donne 2 et 0 ; COUNT(*) donnerait 2 et 1. SUM donne 50 et NULL ; COALESCE(SUM(...), 0) affiche 50 et 0. HAVING COUNT(co.id) >= 2 garde Alice. WHERE intervient avant ce résumé.

Pour approfondir : [31] [42] [38] [37] [41] [33] [40] [34] [43] [32] [39] [35] [36] [44] · Références détaillées à la fin.

SUITE DE LA FICHE

SQL · mémo des étapes et des garanties

Écriture et observation

Outil	Sens à expliquer
CRUD · Create, Read, Update, Delete	Créer, lire, modifier, supprimer : INSERT, SELECT, UPDATE, DELETE.
WHERE / RETURNING	Lignes visées / valeurs renvoyées par l'écriture.
BEGIN / COMMIT / ROLLBACK	Commencer / conserver / annuler la transaction.
\$1 et valeur séparée	Paramétrer une valeur, sans concaténer la saisie au SQL.
Index	Accès parfois facilité, avec coût d'espace et d'écriture.
EXPLAIN / EXPLAIN ANALYZE	Plan estimé / requête réellement exécutée et mesurée.

Les vérifications qui évitent un contresens

- NULL ne se compare pas avec = NULL : employer IS NULL.
- Une ligne conservée par LEFT JOIN n'est pas une commande supplémentaire.
- La FK vérifie une référence, pas le minimum d'enfants du parent.
- Une transaction rend le groupe atomique : tout valider ou tout annuler.
- Un paramètre n'est pas un nom de colonne ; prévoir une liste autorisée.
- Un plan exact ne se devine pas depuis deux lignes ; observer des données représentatives.

Pour approfondir : [31] [42] [38] [37] [41] [33] [40] [34] [43] [32] [39] [35] [36] [44] · Références détaillées à la fin.

Docker - Mémo pour expliquer une commande

Avant une commande, dire si elle agit sur une image, un conteneur, un réseau ou des données.

Les mots du cours

Mot	À expliquer
Image	Paquet préparé avec le programme et son environnement.
Conteneur	Exécution d'une image ; peut être arrêtée.
Dockerfile	Instructions de fabrication de l'image.
Port	Numéro où un programme écoute les connexions.
Réseau Compose	Communication interne par noms de services.
Volume nommé	Stockage indépendant du conteneur ; pas une sauvegarde.

Commande → effet attendu

Commande complète	Effet
<code>docker build -t atelier-api .</code>	Fabriquer l'image depuis le dossier courant.
<code>docker run --name atelier-api atelier-api</code>	Créer un nouveau conteneur et démarrer le programme.
<code>docker stop atelier-api</code>	Arrêter le conteneur sans effacer l'image.
<code>docker start atelier-api</code>	Relancer le conteneur existant.
<code>docker compose up -d</code>	Lancer les services en arrière-plan.
<code>docker compose down</code>	Retirer les conteneurs et le réseau, garder les volumes nommés.

Lire une publication et chercher une erreur

- `-p 127.0.0.1:8080:3000` : mon ordinateur, port 8080 → conteneur, port 3000. EXPOSE documente un port sans le publier.
- Dans `api` : la base se trouve à `db:5432`. Sur l'ordinateur : `localhost:15432` si cette publication existe.
- `docker compose ps` affiche les états ; `docker compose logs api db` lit les messages. La base peut être démarrée sans être encore prête.
- `docker compose down -v` supprime les volumes de ce projet Compose : leurs données sont alors perdues.

Pour approfondir : [09] [06] [07] [08] · Références détaillées à la fin.

SUITE DE LA FICHE

Docker · Mémo pour expliquer une commande

Instructions à lire dans l'ordre

Instruction	Moment et effet
FROM	Choisir l'environnement de base.
WORKDIR	Fixer le dossier de travail.
COPY	Ajouter un fichier dans l'image.
RUN	Exécuter une action pendant la fabrication.
CMD	Choisir la commande de démarrage.

Deux derniers repères

Cache : réutiliser les étapes de fabrication inchangées. .dockerignore : exclure des fichiers du dossier envoyé à Docker, notamment des secrets. Une modification de l'image ne modifie pas un conteneur déjà créé.

Pour approfondir : [09] [06] [07] [08] · Références détaillées à la fin.

Tests JavaScript · Mémo de préparation et de vérification

Écrire la règle et l'attendu avant de choisir la syntaxe de l'outil.

Les mots du cours

Mot	Sens
Assertion	Comparaison entre résultat réel et résultat attendu.
Runner	Lanceur qui exécute les tests et affiche les échecs.
Unitaire	Petite unité de logique ; dépendances contrôlées.
Intégration	Échange réel entre composants.
E2E : end-to-end	Parcours utilisateur de bout en bout.
Fixture / double	Données connues / remplacement contrôlé d'une dépendance.

Choisir une assertion

Vérifier	Node assert	Vitest expect
Une valeur	<code>strictEqual(actual, expected)</code>	<code>expect(actual).toBe(expected)</code>
Un objet ou tableau	<code>deepStrictEqual(actual, expected)</code>	<code>expect(actual).toStrictEqual(expected)</code>
Une exception immédiate	<code>throws(fn)</code>	<code>expect(fn).toThrow()</code>
Une Promise rejetée	<code>await rejects(p)</code>	<code>await expect(p).rejects().toThrow()</code>

Exception immédiate et rejet futur

`fn` désigne une fonction. Une exception immédiate est levée pendant son appel : `throws` reçoit `fn` pour effectuer et contrôler cet appel. `p` désigne une Promise. Un rejet est son résultat d'échec, possiblement futur : `await rejects(p)` attend et vérifie cet échec. Pour une structure, `toStrictEqual` est le rapprochement avec `deepStrictEqual` ; `toEqual` ignore certaines différences, par exemple une propriété `undefined` absente.

Pour approfondir : [17] [16] [29] [15] [14] · Références détaillées à la fin.

SUITE DE LA FICHE

Tests JavaScript · Mémo de préparation et de vérification

erreurs.mjs : exemple Node complet, puis node erreurs.mjs

```
import assert from "node:assert/strict";
const fn = () => { throw new Error("Refus"); };
assert.throws(fn, /Refus/);
const p = Promise.reject(new Error("Refus"));
await assert.rejects(p, /Refus/);
console.log("Deux vérifications réussies");
```

Avant, pendant et après le test

- AAA : Arrange, préparer ; Act, appeler ; Assert, comparer. Donner un nom qui décrit le comportement.
- Cas à examiner : succès, absence, bornes incluses, voisins exclus, mauvais types, erreurs et permissions.
- HTTP, protocole de demande/réponse : vérifier le statut et le corps. fetch peut recevoir 404 sans rejeter sa Promise.
- await attend une opération asynchrone. Attendre aussi les assertions portant sur un rejet.
- Nettoyer les serveurs et connexions : t.after avec Node ; afterEach pour un nettoyage après chaque test avec Vitest.

Lancer une suite dans un projet déjà équipé de l'outil

```
node --test quantite.test.mjs
node --test http.test.mjs
# Avec Vitest installé dans le projet :
npm vitest run
```

Améliorer le code avec les tests

TDD, développement guidé par les tests : Red, test en échec ; Green, code qui le fait passer ; Refactor, amélioration sans changer le comportement. Après un bug corrigé, un test de non-régression vérifie qu'il ne réapparaît pas. La couverture mesure le code parcouru, pas la qualité des attendus.

Pour approfondir : [17] [16] [29] [15] [14] · Références détaillées à la fin.

Backend - Mémo du parcours et des vérifications

Pour chaque demande, expliquer ce que le serveur reçoit, ce qu'il vérifie et ce qu'il renvoie.

Qui fait quoi ?

Rôle	Responsabilité
Route	Associer une méthode HTTP et un chemin au traitement.
Middleware	Préparer, répondre ou transmettre avec next().
Contrôleur	Lire la demande HTTP, adapter les entrées et choisir la réponse.
Service	Appliquer ou orchestrer les règles de traitement.
Repository	Lire et écrire les données.

Statut HTTP : code accompagnant la réponse

Code	Sens dans les contrats du cours
200 / 201	Succès / ressource effectivement créée.
400 / 422	Demande incorrecte ou JSON illisible / contenu compris mais invalide.
401	Identité manquante ou invalide ; inclure WWW-Authenticate.
403 / 404	Permission refusée / ressource absente.
409	Conflit avec l'état actuel, par exemple un doublon interdit.
500	Incident interne ; réponse sans détails secrets.

Trois vérifications différentes

- Validation : la donnée respecte-t-elle le schéma et les règles ? Zod safeParse donne success et, en cas de réussite, data. TypeScript ne valide pas un JSON reçu pendant l'exécution.
- Authentification : quelle identité est établie ? JWT signifie JSON Web Token. Son contenu signé reste lisible. verify contrôle la signature ; impose algorithms, issuer et audience, et exige une expiration selon le contrat. decode ne vérifie pas.
- Autorisation : cette identité a-t-elle le droit d'agir sur cette ressource ? Lire le propriétaire côté serveur ; le client ne crée pas ses droits avec un champ ownerId.

Pour approfondir : [11] [10] [47] [56] [12] [02] [03] [05] [46] [45] · Références détaillées à la fin.

SUITE DE LA FICHE

Backend · Mémo du parcours et des vérifications

Accès aux données

Écriture	Intention
SQL avec \$1 et valeurs séparées	Une saisie reste une valeur, pas du code SQL.
ORM : association objets / tables	Méthodes qui organisent la lecture et l'écriture.
findMany / findUnique	Tableau, même vide / ligne ou null.
where / select / include	Filtrer / choisir les champs / ajouter une relation.
orderBy / skip / take	Ordonner / sauter / limiter des lignes.
Transaction	Valider ensemble ou annuler ensemble des écritures.

Repères à garder après le cours

- express.json avant les routes ; erreurs à quatre paramètres après les routes. Préserver 400 pour JSON mal formé ; 500 pour un incident sans statut attendu.
- Prisma 6.19.0 : migrate dev crée et applique en développement ; migrate deploy applique au déploiement ; generate fabrique le client sans créer les tables.
- Hachage de mot de passe adapté et sel aléatoire individuel, pas de texte clair. Cookie : HttpOnly empêche la lecture JavaScript ; Secure limite l'envoi à HTTPS.
- CSRF : falsification de requête intersite utilisant notamment un cookie envoyé automatiquement. Protéger les actions sensibles ; SameSite complète la protection.
- Une interface masquée ne protège pas une API ; vérifier les permissions côté serveur à chaque opération.

Pour approfondir : [11] [10] [47] [56] [12] [02] [03] [05] [46] [45] · Références détaillées à la fin.

React · mémo du trajet entre donnée et écran

Avant la syntaxe, identifier la donnée, son propriétaire et ce qui doit déclencher une nouvelle description de l'écran.

Vocabulaire développé

Notion	Sens déjà démontré
JSX	Syntaxe d'interface dans JavaScript ; {expression} insère une valeur.
Composant / props, properties	Fonction d'interface / entrées fournies par le parent.
State, état / render, rendu	Mémoire React / calcul de la description de l'écran.
Événement / gestionnaire	Action signalée / fonction appelée pour y répondre.
useState / updater	Obtenir l'état / calculer le suivant depuis le précédent.
Champ contrôlé	value lit l'état ; onChange demande sa mise à jour.

Trajet d'une saisie

Alice est saisi → onChange reçoit l'événement → event.target.value contient Alice → setNom demande le nouvel état → React rend à nouveau → value et l'aperçu lisent Alice. Pour un clic, onClick reçoit une fonction sans l'appeler pendant le rendu. Un updater comme $n \Rightarrow n + 1$ convient quand le prochain état dépend du précédent.

Responsabilités et synchronisation

Outil	Question à poser
État partagé	Quel parent commun conserve la seule valeur de référence ?
map / key	Transformer les éléments / conserver leur identité stable.
Nouvelle liste / filter	Produire une valeur sans modifier directement l'ancien tableau.
useEffect / dépendances	Quelle ressource extérieure synchroniser et quelles valeurs sont lues ?
Nettoyage	Que faut-il terminer, retirer ou rendre inactif ?
Promise / async / await	Résultat futur / fonction qui le retourne / attendre son résultat.

Trois protections de compréhension

Le JSX appartient au rendu ; le return d'un effet peut fournir un nettoyage, pas un écran. L'état de chargement n'est ni un succès vide ni une erreur. Une réponse ancienne ne doit plus modifier l'état après nettoyage : tester son actif protège le résultat actuel. L'effet retourne ce nettoyage ; sa fonction async interne utilise await et traite le rejet.

Pour approfondir : [18] [23] [21] [19] [20] [22] [25] [26] [24] · Références détaillées à la fin.

SUITE DE LA FICHE

React · mémo du trajet entre donnée et écran

Navigation développée

Mot	Lecture concrète
URL, adresse	/activites/lire est le chemin demandé.
BrowserRouter / Routes / Route	Contexte navigateur / règles / chemin et écran associé.
Paramètre / useParams	/activites/:id donne ici le texte id = "lire".
Navigate	Demander une redirection selon une règle connue.

Les vérifications finales

- Un simple texte calculé depuis les props ne nécessite pas d'effet.
- Une prop est une entrée ; l'enfant ne la modifie pas.
- Une key est stable malgré une suppression ou un changement d'ordre.
- Une dépendance suit les valeurs réactives réellement utilisées.
- Le routeur choisit un écran ; les permissions sont vérifiées côté serveur.

Pour approfondir : [18] [23] [21] [19] [20] [22] [25] [26] [24] · Références détaillées à la fin.

Lire les références

Les situations sont créées pour apprendre. Les liens ci-dessous donnent les références des notions : documentation des outils, spécification UML ou travaux académiques selon le sujet.

[01] Agile Alliance · Modèle de user story

Documentation officielle

<https://agilealliance.org/glossary/user-story-template/>

[02] OWASP · vérifier les permissions

Documentation officielle

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[03] OWASP · stockage des mots de passe

Documentation officielle

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[04] Sibel Adali, RPI · Dépendances, formes normales et décomposition

Référence académique

https://cs.rpi.edu/~sibel/csci4380/fall2020/course_notes/normalization.html

[05] MDN · propriétés d'un cookie HTTP

Documentation officielle

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>

[06] Docker Compose · réseau et noms de services

Documentation officielle

<https://docs.docker.com/compose/how-tos/networking/>

[07] Docker · volumes persistants

Documentation officielle

<https://docs.docker.com/engine/storage/volumes/>

[08] Docker Compose · arrêt et suppression

Documentation officielle

<https://docs.docker.com/reference/cli/docker/compose/down/>

[09] Docker · instructions du Dockerfile

Documentation officielle

<https://docs.docker.com/reference/dockerfile/>

[10] Express 5 · traitement des erreurs

Documentation officielle

<https://expressjs.com/en/guide/error-handling/>

[11] Express 5 · chaîne de middlewares

Documentation officielle

<https://expressjs.com/en/guide/using-middleware/>

[12] jsonwebtoken · signature, verify et decode

Documentation officielle

<https://github.com/auth0/node-jwt-token>

Références · suite

[13] Mocodo · Manuel de référence de l'auteur

Documentation officielle

https://laowantong.github.io/mocodo/doc/fr_refman.html

[14] Microsoft Learn · cycle Red, Green, Refactor

Documentation officielle

<https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/contact-manager/iteration-6-use-test-driven-development-cs>

[15] Microsoft Learn · organiser des tests unitaires avec Arrange, Act, Assert

Documentation officielle

<https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices>

[16] Node.js · vérifier un résultat avec assert

Documentation officielle

<https://nodejs.org/api/assert.html>

[17] Node.js · exécuter des tests

Documentation officielle

<https://nodejs.org/api/test.html>

[18] React · Transmettre des props

Documentation officielle

<https://react.dev/learn/passing-props-to-a-component>

[19] React · Partager un état

Documentation officielle

<https://react.dev/learn/sharing-state-between-components>

[20] React · Mettre à jour un tableau

Documentation officielle

<https://react.dev/learn/updating-arrays-in-state>

[21] React · Champs input

Documentation officielle

<https://react.dev/reference/react-dom/components/input>

[22] React · useEffect et nettoyage

Documentation officielle

<https://react.dev/reference/react/useEffect>

[23] React · useState

Documentation officielle

<https://react.dev/reference/react/useState>

[24] React Router · Navigate

Documentation officielle

<https://reactrouter.com/api/components/Navigate>

[25] React Router · BrowserRouter

Documentation officielle

<https://reactrouter.com/api/declarative-routers/BrowserRouter>

Références · suite

[26] React Router · useParams

Documentation officielle

<https://reactrouter.com/api/hooks/useParams>

[27] Stéphane Crozat · Théorie de la normalisation relationnelle

Référence académique

<https://stph.scenari-community.org/bdd/nor1.pdf>

[28] tsx · Exécution et vérification des types

Documentation officielle

<https://tsx.is/typescript>

[29] Vitest · comparer les résultats

Documentation officielle

<https://vitest.dev/api/expect.html>

[30] OMG · UML 2.5.1, standard et document normatif, chapitre 18

Norme / spécification

<https://www.omg.org/spec/UML/2.5.1>

[31] PostgreSQL · Contraintes

Documentation officielle

<https://www.postgresql.org/docs/current/ddl-constraints.html>

[32] PostgreSQL · RETURNING

Documentation officielle

<https://www.postgresql.org/docs/current/dml-returning.html>

[33] PostgreSQL · Comparaisons et NULL

Documentation officielle

<https://www.postgresql.org/docs/current/functions-comparison.html>

[34] PostgreSQL · COALESCE et expressions conditionnelles

Documentation officielle

<https://www.postgresql.org/docs/current/functions-conditional.html>

[35] PostgreSQL · Sous-requêtes

Documentation officielle

<https://www.postgresql.org/docs/current/functions-subquery.html>

[36] PostgreSQL · Index

Documentation officielle

<https://www.postgresql.org/docs/current/indexes.html>

[37] PostgreSQL · LIMIT et OFFSET

Documentation officielle

<https://www.postgresql.org/docs/current/queries-limit.html>

[38] PostgreSQL · ORDER BY

Documentation officielle

<https://www.postgresql.org/docs/current/queries-order.html>

Références · suite

[39] PostgreSQL · PREPARE et paramètres

Documentation officielle

<https://www.postgresql.org/docs/current/sql-prepare.html>

[40] PostgreSQL · Agrégats

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-agg.html>

[41] PostgreSQL · Jointures

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-join.html>

[42] PostgreSQL · Interroger une table

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-select.html>

[43] PostgreSQL · Transactions

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-transactions.html>

[44] PostgreSQL · EXPLAIN

Documentation officielle

<https://www.postgresql.org/docs/current/using-explain.html>

[45] Prisma ORM 6 · migrations en développement et déploiement

Documentation officielle

<https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/development-and-production>

[46] Prisma ORM 6 · méthodes du client

Documentation officielle

<https://www.prisma.io/docs/orm/v6/reference/prisma-client-reference>

[47] RFC 9110 · codes de réponse HTTP

Norme / spécification

<https://www.rfc-editor.org/rfc/rfc9110.html#name-status-codes>

[48] TypeScript · Bases et compilateur

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/basic-types.html>

[49] TypeScript · Types courants

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

[50] TypeScript · Fonctions

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/functions.html>

[51] TypeScript · Génériques

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

Références · suite

[52] TypeScript · Modules et imports de types

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/modules.html>

[53] TypeScript · Narrowing et unions discriminées

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html>

[54] TypeScript · Types utilitaires

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/utility-types.html>

[55] TypeScript · Référence tsconfig

Documentation officielle

<https://www.typescriptlang.org/tsconfig/>

[56] Zod · validation des données à l'exécution

Documentation officielle

<https://zod.dev/basics>